

实验一 1-3 学时实践记录（提交汇编稿）

成员：20241328 蔡贺俊（Alice）、20241304 元泓鉴（Bob）

环境：openEuler 24.03 LTS / OpenSSL 3.0.12 / GmSSL 3.3.0-dev

环境准备（蔡贺俊）

工具链与算法库版本

```
$ gcc --version | head -1
gcc (GCC) 12.3.1 (openEuler 12.3.1-38.oe2403)
$ cmake --version | head -1
cmake version 3.27.9
$ openssl version
OpenSSL 3.0.12 24 Oct 2023 (Library: OpenSSL 3.0.12 24 Oct 2023)
$ gmssl version
GmSSL 3.3.0-dev.1183
$ cat /etc/ld.so.conf.d/gmssl.conf
/usr/local/lib
```

准备过程

1. WSL2 安装 openEuler 24.03 LTS（作业推荐系统）。
2. `dnf install -y gcc-c++ cmake openssl-devel`（OpenSSL 开发头文件与编译工具）。
3. 源码编译 GmSSL 3.3.0: `git clone https://github.com/guanzhi/GmSSL.git` → `cmake -B build && cmake --build build -j` → `cmake --install build`, 构建全部目标成功（100% Built target ...）。
4. 配置 `echo /usr/local/lib > /etc/ld.so.conf.d/gmssl.conf && ldconfig`, 解决 `libgmssl` 链接与运行时的库路径问题（详见问题与反思第1条）。

任务1 OpenSSL 命令实践（蔡贺俊）

环境：openEuler 24.03 LTS（WSL2）、OpenSSL 3.0.12

工作目录：~/work/task1（实践产物不入仓库，仅记录入库）

1. 版本查询、随机数与 SM4-CBC 对称加解密

```
$ openssl version -a | head -4
OpenSSL 3.0.12 24 Oct 2023 (Library: OpenSSL 3.0.12 24 Oct 2023)
built on: Tue Apr 21 09:41:41 2026 UTC
platform: linux-x86_64
options: bn(64,64)

$ printf 'Hello 密码系统设计 20241328 蔡贺俊\n' > plain.txt
$ openssl rand -hex 16 | tee sm4.key
25d20bb20a3d1a6d355f1a30959d4c73

$ openssl enc -sm4-cbc -e -in plain.txt -out plain.sm4 -K $(cat sm4.key) -iv 0000000000000000
0000000000000000
$ openssl enc -sm4-cbc -d -in plain.sm4 -out plain.dec -K $(cat sm4.key) -iv 0000000000000000
0000000000000000
```

```
$ cmp plain.txt plain.dec && echo SM4-CBC-ROUNDTRIP-OK
SM4-CBC-ROUNDTRIP-OK
```

```
$ od -A x -t x1 plain.sm4 | head -2
000000 69 0a b7 3d 8c f6 3a 34 77 90 d4 88 3f ea 2c c9
000010 c2 ad b4 64 14 0e 12 bb b8 95 61 10 b2 f7 09 a4
```

说明：明文为姓名+学号；SM4 密钥用 `openssl rand -hex 16` 产生 16 字节；IV 为 16 字节全 0（演示用）；加解密往返逐字节一致。

2. SM3 摘要与 HMAC

```
$ openssl dgst -sm3 plain.txt
SM3(plain.txt)= 4543fcd70da081d21d1310e973e4adcb94dce80b3cc6b21051c630740c913cc2

$ openssl dgst -sm3 -hmac mykey123 plain.txt
HMAC-SM3(plain.txt)= 6dfb23c9f1d0f62fe7008353a13b4326c7187f39e84ae542d8204a8a487abbc9

$ { printf 'X'; tail -c +2 plain.txt; } > plain2.txt # 仅改第1字节 H->X
$ cmp -l plain.txt plain2.txt | head -1
1 110 130

$ openssl dgst -sm3 plain2.txt
SM3(plain2.txt)= 00841b73cac3992030440ade876be65bab0145dea1882506376d614477685c24
```

说明：仅改动 1 个字节后摘要值完全改变，验证 SM3 雪崩效应；HMAC-SM3 以密钥 mykey123 做消息认证码。

3. SM2 密钥生成、签名与验签

```
$ openssl ecparam -list_curves | grep -i sm2
SM2      : SM2 curve over a 256 bit prime field

$ openssl ecparam -genkey -name SM2 -out sm2_priv.pem
$ openssl ec -in sm2_priv.pem -pubout -out sm2_pub.pem

read EC key
-----BEGIN PUBLIC KEY-----
MFowFAYIKoEcz1UBgiOGCCqBHM9VAYItA0IABDF0erAAQhZwhNr2dJ3sPkVxTpj4
(以下略)

writing EC key
$ openssl dgst -sm3 -sign sm2_priv.pem -out sig.bin plain.txt
$ openssl dgst -sm3 -verify sm2_pub.pem -signature sig.bin plain.txt
Verified OK
```

说明：OpenSSL 3.0 原生支持 SM2 曲线；SM2 签名默认以 SM3 为摘要算法（签名者 ID 取默认值 1234567812345678）；验签输出 Verified OK。sm2_priv.pem 仅存实践目录，未入库。

4. speed 性能测试

```
$ openssl speed -seconds 1 -bytes 16384 -evp sm4-cbc
The 'numbers' are in 1000s of bytes per second processed.
type                16384 bytes
SM4-CBC              112172.48k

$ openssl speed -seconds 1 -evp sm3
type                16 bytes      64 bytes      256 bytes    1024 bytes    8192 bytes    16384 bytes
sm3                 56400.90k    133060.42k    243829.50k    261999.62k    320790.53k    281313.28k
```

说明：SM4-CBC 大块吞吐约 110 MB/s；SM3 在 8KB 块上约 320 MB/s（WSL2 虚拟机单线程实测，仅作参考）。

小结

任务1 完成 OpenSSL 命令实践四项：对称加解密（SM4-CBC）、摘要与 HMAC（SM3）、非对称签名验签（SM2）、性能测试（speed），全部在 openEuler 24.03 上验证通过。

任务1：OpenSSL 命令实践记录

实验人

20241304 元泓鉴

环境

WSL openEuler 24.03, OpenSSL 3.0.12

操作内容

SM3 摘要

```
openssl dgst -sm3 plain.txt
```

对明文文件计算 SM3 杂凑值，输出32字节摘要。

HMAC-SM3

```
openssl dgst -sm3 -hmac "mykey123" plain.txt
```

使用密钥计算带消息验证码的 SM3 值。

SM4-CBC 加解密

```
openssl rand -hex 16
```

```
openssl enc -sm4-cbc -e -K <key> -iv <iv> -in plain.txt -out enc.bin
```

```
openssl enc -sm4-cbc -d -K <key> -iv <iv> -in enc.bin
```

生成随机16字节密钥，使用 SM4-CBC 模式加解密，PKCS#7 填充。

SM2 密钥生成

```
openssl ecparam -genkey -name SM2 -out sm2_priv.pem
```

```
openssl pkey -in sm2_priv.pem -pubout -out sm2_pub.pem
```

SM2 加解密

```
openssl pkeyutl -encrypt -pubin -inkey sm2_pub.pem -in k.bin -out KC.bin
```

```
openssl pkeyutl -decrypt -inkey sm2_priv.pem -in KC.bin -out k_recv.bin
```

SM2 签名验签

```
openssl dgst -sm3 -sign sm2_priv.pem -out sign.bin plain.txt
```

```
openssl dgst -sm3 -verify sm2_pub.pem -signature sign.bin plain.txt
```

结果

全部操作成功，加解密往返一致，验签输出 Verified OK。

任务2 GmSSL 命令实践（蔡贸俊）

环境：openEuler 24.03 LTS（WSL2）、GmSSL 3.3.0-dev.1183（源码编译安装）

工作目录：~/work/task2

1. 版本、随机数与 SM4-CBC 加解密

```
$ gmssl version
GmSSL 3.3.0-dev.1183

$ gmssl rand -hex -outlen 16 | tee sm4.key
E11DD1C88E2355692465180DCFFDD668C

$ gmssl sm4_cbc -encrypt -pkcs7_padding -key $(cat sm4.key) -iv 0000000000000000000000000000
0000 -in plain.txt -out plain.sm4
$ gmssl sm4_cbc -decrypt -pkcs7_padding -key $(cat sm4.key) -iv 0000000000000000000000000000
0000 -in plain.sm4 -out plain.dec
$ cmp plain.txt plain.dec && echo GmSSL-SM4-ROUNTRIP-OK
GmSSL-SM4-ROUNTRIP-OK

$ od -A x -t x1 plain.sm4 | head -2
000000 ed 45 f0 e3 20 53 84 75 4c c2 74 68 86 1a b1 88
000010 ec 87 c6 9f 48 9a 17 e6 dc 7b 09 0a fb 3e 57 2f
```

说明：GmSSL 的 rand 用 -outlen 指定字节数；sm4_cbc 对非 16 字节倍数明文必须显式加 -pkcs7_padding（首次运行报 input length must be multiple of 16 bytes，加该选项后通过）；密文 48 字节 = 45 字节明文 + 3 字节 PKCS#7 填充。

2. SM3 摘要、HMAC 与 OpenSSL 交叉验证

```
$ gmssl sm3 -in plain.txt
4543fcd70da081d21d1310e973e4adcb94dce80b3cc6b21051c630740c913cc2

$ gmssl sm3 -in ../task1/plain2.txt
00841b73cac3992030440ade876be65bab0145dea1882506376d614477685c24

$ gmssl sm3_hmac -key 6d796b65793132336d796b6579313233 -in plain.txt
b8e7ce80387a45a853065ba2efc0b4024a7e95da8e1b36bf073d32c8ealb67b7
```

说明：GmSSL 的 SM3 摘要与任务1中 OpenSSL openssl dgst -sm3 的结果逐字节一致（4543fcd7... 与 00841b73...），两套实现交叉验证正确；sm3_hmac 要求密钥至少 12 字节（首次用 8 字节密钥报 key should be at least 24 digits，改 16 字节后通过）。

3. SM2 密钥生成、签名验签、加密解密

```
$ gmssl sm2keygen -pass 123456 -out sm2_priv.pem -pubout sm2_pub.pem
$ gmssl sm2sign -key sm2_priv.pem -pass 123456 -in plain.txt -out sig.bin
$ gmssl sm2verify -pubkey sm2_pub.pem -in plain.txt -sig sig.bin
verify : success
$ gmssl sm2encrypt -pubkey sm2_pub.pem -in plain.txt -out sm2.enc
```

```
$ gmssl sm2decrypt -key sm2_priv.pem -pass 123456 -in sm2.enc -out sm2.dec
$ cmp plain.txt sm2.dec && echo GMSSL-SM2-ENC-ROUNDTRIP-OK
GMSSL-SM2-ENC-ROUNDTRIP-OK
```

说明：GmSSL 私钥以口令加密保存（签名/解密需 `-pass`）；`sm2verify` 的公钥参数为 `-pubkey`（首次误用 `-pkey` 报错，查 `usage` 后修正）；签名验签、加密解密均通过。

小结

完成 GmSSL 命令实践：SM4-CBC 加解密、SM3 摘要/HMAC、SM2 密钥生成/签名验签/加密解密，并与 OpenSSL 的 SM3 摘要交叉验证一致。踩坑两条（`sm4_cbc` 需显式 `-pkcs7_padding`、`sm3_hmac` 密钥长度下限）已记入问题与反思素材。

任务2：GmSSL 命令实践记录

实验人

20241304 元泓鉴

环境

WSL openEuler 24.03, GmSSL 3.3.0-dev.1183（源码编译安装至 `~/.local/gmssl`）

操作内容

SM3 摘要

```
gmssl sm3 plain.txt
```

计算结果与 OpenSSL `sm3` 一致，验证正确性。

SM4-CBC 加解密

```
gmssl sm4_cbc -encrypt -key <key> -iv <iv> -pkcs7_padding -in plain.txt -out C.bin
gmssl sm4_cbc -decrypt -key <key> -iv <iv> -pkcs7_padding -in C.bin -out plain_recv.txt
```

SM2 密钥生成

```
gmssl sm2keygen -pass 12345678 -out sm2_priv.pem -pubout sm2_pub.pem
```

SM2 加解密

```
gmssl sm2encrypt -pubkey sm2_pub.pem -in k.bin -out KC.bin
gmssl sm2decrypt -pass 12345678 -inkey sm2_priv.pem -in KC.bin -out k_recv.bin
```

SM2 签名验签

```
gmssl sm2sign -key sm2_priv.pem -pass 12345678 -in C.bin -out S1.bin
gmssl sm2verify -pubkey sm2_pub.pem -signature S1.bin -in C.bin
```

结果

全部操作成功，GmSSL 与 OpenSSL 的 SM3 摘要值完全一致，交叉验证通过。

任务3 OpenSSL 命令数字信封 Alice→Bob

角色: Alice=20241328 蔡贤俊 (发送), Bob=20241304 元泓鉴 (接收)

密钥: Alice 公钥 keys/20241328_caimaojun/sm2_openssl_pub.pem (私钥本机); Bob 公钥 keys/20241304_yuanhongjian/sm2_openssl_pub.pem

传递文件: exchange/task3/ (C.bin、KC.bin、S1.bin、envelope.bin、lens.txt)

Alice 发送侧

```
$ printf '蔡贤俊 20241328' > plain.txt
$ gmssl rand -hex -outlen 16 | tr -d '\n' | tr 'A-F' 'a-f' > k.hex
093f3fc30861d9daa78415b1ddb49c58
$ python3 -c 'import binascii,sys;sys.stdout.buffer.write(binascii.unhexlify(sys.stdin.read(
)).strip()))' < k.hex > k.bin
$ openssl enc -sm4-cbc -e -in plain.txt -out C.bin -K $(cat k.hex) -iv 00000000000000000000
000000000000
$ openssl pkeyutl -encrypt -pubin -inkey keys/20241304_yuanhongjian/sm2_openssl_pub.pem -in
k.bin -out KC.bin
$ openssl dgst -sm3 -sign alice_openssl_priv.pem -out S1.bin C.bin
$ openssl dgst -sm3 -verify alice_openssl_pub.pem -signature S1.bin C.bin
Verified OK
$ openssl enc -sm4-cbc -d -in C.bin -out selfcheck.txt -K $(cat k.hex) -iv 00000000000000000000
0000000000000000
$ cmp plain.txt selfcheck.txt && echo SELF-CHECK-OK
SELF-CHECK-OK
$ cat C.bin KC.bin S1.bin > envelope.bin
$ stat -c '%n %s' C.bin KC.bin S1.bin
C.bin 32
KC.bin 122
S1.bin 71
```

协议对应: $k = \text{gmssl rand}$

16字节; $C = \text{Sm4Enc}(k, P)$; $KC = \text{Sm2Enc}(PK_b, k)$; $S1 = \text{Sm2Sign}(SK_a, C)$; 数字信封 = $C || KC || S1$ 。

发送侧自验: 用自己的公钥验 S1 通过、用 k 解 C 与明文逐字节一致后才发布。

坑: openssl dgst 不接受 -in 参数, 待签名/验签文件必须用位置参数给出 (误用 -in 报 Can only sign or verify one file)。

Bob 接收侧 (元泓鉴填写)

(待填: $\text{Sm2Verify}(PK_a, S1) \rightarrow \text{Sm2Dec}(SK_b, KC) = k \rightarrow \text{Sm4Dec}(k, C) = P$, 明文应为"蔡贤俊 20241328")

Bob 接收侧 (元泓鉴 20241304)

环境

- WSL openEuler 24.03, OpenSSL 3.0.12
- Bob 私钥: openssl-cmd/sm2_priv.pem
- Alice 公钥: keys/20241328_caimaojun/sm2_openssl_pub.pem

接收三步

1. 验签

```
openssl dgst -sm3 -verify sm2_openssl_pub.pem -signature S1.bin C.bin
```

结果: Verified OK

2. SM2 解密 KC

```
openssl pkeyutl -decrypt -inkey sm2_priv.pem -in KC.bin -out k.bin
```

恢复的 SM4 密钥 (hex) : 093f3fc30861d9daa78415b1ddb49c58 (16字节)

3. SM4-CBC 解密 C

```
openssl enc -sm4-cbc -d -in C.bin -K 093f3fc30861d9daa78415b1ddb49c58 -iv 00000000000000000000000000000000
```

明文: 蔡贸俊 20241328

结论

数字信封接收端完整跑通: 验签通过、密钥恢复正确、SM4 解密得到预期明文。

Bob 接收侧 (元泓鉴 20241304)

命令版数字信封接收 (exchange/task3/)

1. 验签:

```
openssl dgst -sm3 -verify sm2_openssl_pub.pem -signature S1.bin C.bin
```

结果: Verified OK

2. SM2 解密 KC:

```
openssl pkeyutl -decrypt -inkey sm2_priv.pem -in KC.bin -out k.bin
```

恢复密钥 hex: 093f3fc30861d9daa78415b1ddb49c58

3. SM4-CBC 解密 C:

```
openssl enc -sm4-cbc -d -in C.bin -K 093f3fc30861d9daa78415b1ddb49c58 -iv 00000000000000000000000000000000
```

明文: 蔡贸俊 20241328

库版数字信封接收 (exchange/task3_lib/)

1. 验签: Verified OK

2. SM2 解密 KC 恢复密钥: 83773f00767a2119a90ea6700e9239c4

3. SM4 解密 C 明文: 蔡贸俊 20241328

结论

命令版和库版数字信封接收均成功, 验签通过、密钥恢复正确、解密得到预期明文。

任务4 GmSSL 命令数字信封 Bob→Alice

角色: Bob=20241304 元泓鉴 (发送), Alice=20241328 蔡贸俊 (接收)

密钥: Bob 公钥 keys/20241304_yuanhongjian/sm2_gmssl_pub.pem; Alice 私钥本机 (GmSSL 格式)

传递文件: exchange/task4/ (C.bin、KC.bin、S1.bin)

Alice 接收侧

```
$ gmssl sm2verify -pubkey keys/20241304_yuanhongjian/sm2_gmssl_pub.pem -in exchange/task4/C.  
bin -sig exchange/task4/S1.bin
```

```

verify : success
$ gmssl sm2decrypt -key alice_gmssl_priv.pem -pass ***** -in exchange/task4/KC.bin -out k4.
bin
$ python3 -c '...hexlify...' < k4.bin
7867646ecd25d0ff82b00cee25975aae
$ gmssl sm4_cbc -decrypt -pkcs7_padding -key 7867646ecd25d0ff82b00cee25975aae -iv 0000000000
000000000000000000000000 -in exchange/task4/C.bin
(首块乱码, 第二块出现合法 PKCS#7 填充 0x0d x13)

```

问题: 全 0 IV 解密首块乱码, 但第二块填充合法 → 密钥与 CBC+PKCS#7 模式正确, 发送方使用了非零随机 IV 且未随信封传递。

处理 (已知明文恢复 IV) :

- 第二块明文末 3 字节 0x89 0xB4 0x0a = "鉴"末两字节+换行 → 推断明文为"20241304 元泓鉴\n"共 19 字节, 首块明文 P1 = 32 30 32 34 31 33 30 34 20 e5 85 83 e6 b3 93 e9
- CBC 性质: $P1 = D(K, C1) \text{ xor } IV$, 故 $IV = D(K, C1) \text{ xor } P1$
- 无填充解密取首块 $D(K, C1) = 95a57c1cbf2a25e7a2a434c6c3878775$
- 得 $IV = a7954e288e1915d38241b1452534149c$

```

$ gmssl sm4_cbc -decrypt -pkcs7_padding -key 7867646ecd25d0ff82b00cee25975aae -iv a7954e288e
1915d38241b1452534149c -in exchange/task4/C.bin
20241304 元泓鉴

```

结论: $Sm2Verify(PKa, S1)$ 通过、 $Sm2Dec(SKa, KC)=k$ 、 $Sm4Dec(k, C)=P$ ="20241304 元泓鉴", 接收侧完成。

反思: 数字信封协议应约定 IV 传递方式 (随信封头部携带或固定全 0); 本次接收方以已知明文 (对方学号姓名) 恢复 IV, 属协议设计教训, 已留言请 Bob 补交 iv.bin。

Bob 发送侧 (元泓鉴填写)

(待填: k 生成、 $C=Sm4Enc(k, P)$ 、 $KC=Sm2Enc(PKa, k)$ 、 $S1=Sm2Sign(SKb, C)$ 的命令与输出, 含所用 IV)

Bob 发送侧 (元泓鉴 20241304)

密钥生成:

```

$ gmssl rand -outlen 16 -hex
KEY=7867646ECD25D0FF82B00CEE25975AAE
IV=A7954E288E1915D38241B1452534149C

```

SM4 加密明文:

```

$ gmssl sm4_cbc -encrypt -key $KEY -iv $IV -pkcs7_padding -in plain.txt -out C.bin
C.bin = 32 bytes

```

SM2 加密密钥 (Alice公钥) :

```

$ printf '\x78\x67\x64\x6e\xcd\x25\xd0\xff\x82\xb0\x0c\xee\x25\x97\x5a\xae' > k.bin
$ gmssl sm2encrypt -pubkey alice_pub.pem -in k.bin -out KC.bin
KC.bin = 122 bytes

```

SM2 签名密文 (Bob私钥) :

```

$ gmssl sm2sign -key bob_priv.pem -pass 12345678 -in C.bin -out S1.bin
S1.bin = 72 bytes

```

发送文件:

C.bin、KC.bin、S1.bin 发布至 exchange/task4/。

说明: 首次发送时 IV 使用随机值且未随信封传递, 后补交 exchange/task4/iv.bin (十六进制 A7954E288E1915D38241B1452534149C)。后续编程版信封统一使用全0 IV。